

DFSha: Especificación del proyecto

Hito 1 | ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos, 2026-2
| Universidad EAFIT

Andrés Vélez Rendón

Jose Luis Restrepo

13 de septiembre de 2026

Índice general

1. Definición del servicio	2
Componentes	2
Operaciones del cliente	2
Cómo se cubren los requisitos no funcionales	3
Fuera del alcance	4
2. Arquitectura del sistema	4
Parámetros configurables	5
Cómo se parte un archivo	6
Cuántas copias se guardan	7
Cómo se decide en qué nodo va cada bloque	7
Quién hace la partición	7
Qué pasa al agregar un nodo	7
Qué pasa al quitar un nodo	8
Qué pasa si se cae el ControlNode	8
Cómo se mantiene la consistencia	8
3. Comunicaciones de red	9
Protocolo por cada relación	9
API del ControlNode, la consume el cliente	10
API del DataNode	11
API interna del clúster	11
Cómo funciona una subida	12
Cómo funciona una descarga	12
5. Infraestructura y entorno	12
Notas	13

Opción asignada: Opción 1, Cliente/Servidor, con el servicio distribuido entre varios nodos dentro de una red privada.

Las decisiones se enuncian en el cuerpo y se justifican en las **Notas** del final.

1. Definición del servicio

DFSha es un sistema de archivos distribuido **por bloques**. Guarda archivos grandes partiéndolos en bloques que se reparten entre varios servidores, y los recupera reconstruyéndolos a partir de esos bloques.

La idea central es que un archivo grande nunca se guarda completo en una sola máquina ni se carga completo en memoria. Se parte, se reparte, y de cada pedazo se guarda más de una copia, para que la caída de un servidor no signifique perder el archivo.

Se escoge el enfoque por bloques y no por objetos porque el RNF4 pide que *cada archivo* esté distribuido entre varios nodos, y el RNF5 pide concurrencia en el acceso a los datos *dentro* de un archivo. Con el archivo como unidad de distribución ninguna de las dos cosas es posible [N1].

El servicio es agnóstico al contenido: guarda ejecutables, binarios, videos, respaldos o volcados de bases de datos sin interpretarlos. Un archivo de video se trata igual que cualquier otro, y el sistema no reproduce ni transmite nada.

Componentes

- **Cliente:** programa de línea de comandos. Se autentica, ejecuta los comandos del sistema de archivos, parte los archivos al subirlos y los reconstruye al bajarlos. No forma parte del servicio.
- **ControlNode:** guarda los metadatos, es decir qué bloques tiene cada archivo, dónde está cada bloque, de quién es cada archivo y qué nodos están vivos. Además decide dónde se ubica cada bloque. No guarda datos de archivos.
- **DataNode:** guarda los bloques en su disco local y los entrega cuando se los piden.

Cada usuario tiene su propio árbol de directorios. Un archivo pertenece a quien lo creó y solo él puede leerlo, borrarlo o moverlo, salvo que conceda permiso explícito sobre un directorio.

Operaciones del cliente

La interfaz es de línea de comandos, que es la forma natural de exponer una jerarquía de archivos y la que permite mostrar en los registros el particionamiento y la ubicación de cada bloque.

RF1. Gestión del sistema de archivos

Comando	Qué hace
ls	Lista el contenido de un directorio
cd	Cambia de directorio. Lo maneja el cliente y solo consulta al servicio para validar que la ruta existe
mkdir / rmdir	Crea un directorio, o borra uno vacío
rm	Borra un archivo y sus bloques
mv	Renombra o mueve, cambiando solo metadatos
stat	Muestra tamaño, número de bloques y copias vivas

RF2. Transferencia de archivos

Comando	Qué hace
put <local> <remoto> (send)	Parte el archivo en bloques y los distribuye entre los DataNodes
get <remoto> <local> (receive)	Recoge los bloques de las réplicas disponibles y reconstruye el archivo

RF3. Acceso a archivos

Transferencia y acceso son servicios distintos. De cara al usuario, DFSha transfiere archivos completos. Por debajo, entre cliente y DataNode, trabaja a nivel de bloque, y sobre ese nivel se definen las primitivas:

Primitiva	Qué hace en DFSha
open	Pide al ControlNode la lista de bloques y devuelve un identificador de sesión. No mueve datos
read	Traduce un rango de bytes a los bloques que lo contienen y descarga solo esos, lo que permite leer parte de un archivo sin bajarlo entero
write	Acumula bytes hasta completar un bloque y lo envía al DataNode asignado
lock	Reserva el archivo para un solo escritor mientras dura la subida
close	Cierra la sesión. En escritura, avisa que la subida terminó

Así, put es open más varios write más close, y get es open más varios read más close.

Cómo se cubren los requisitos no funcionales

	Requisito	Cómo se cubre
RNF1	Escalabilidad	La capacidad es la suma de los discos de los DataNodes y se crece agregando nodos sin detener el servicio. El ControlNode solo guarda metadatos, despreciables frente a los datos [N2]
RNF2	Alta disponibilidad	Cada bloque vive en varios DataNodes, con el número de copias derivado de un objetivo de disponibilidad [N4]. El ControlNode está replicado y sus metadatos son persistentes [N7]

	Requisito	Cómo se cubre
RNF3	Consistencia	Escritura única: los bloques no se modifican nunca, y un archivo no aparece hasta que todos sus bloques terminaron de subirse [N8]
RNF4	Particionamiento	Todo archivo se parte en bloques repartidos entre nodos, y las copias de un mismo bloque nunca quedan en el mismo DataNode [N3]
RNF5	Rendimiento	El cliente transfiere varios bloques en paralelo contra DataNodes distintos, y cada DataNode atiende varias peticiones a la vez [N10]
RNF6	Seguridad	HTTPS en todos los canales y cifrado de cada bloque en disco. Control de acceso por usuario. Entre nodos, autenticación con una clave compartida del clúster, de modo que un nodo ajeno no puede registrarse ni recibir órdenes
RNF7	Transparencia	El cliente no tiene direcciones de DataNodes en su configuración: las recibe en cada operación, de forma dinámica y nunca estática [N6]
RNF8	Ajustes	Se documentará cualquier cambio en los hitos siguientes

Fuera del alcance

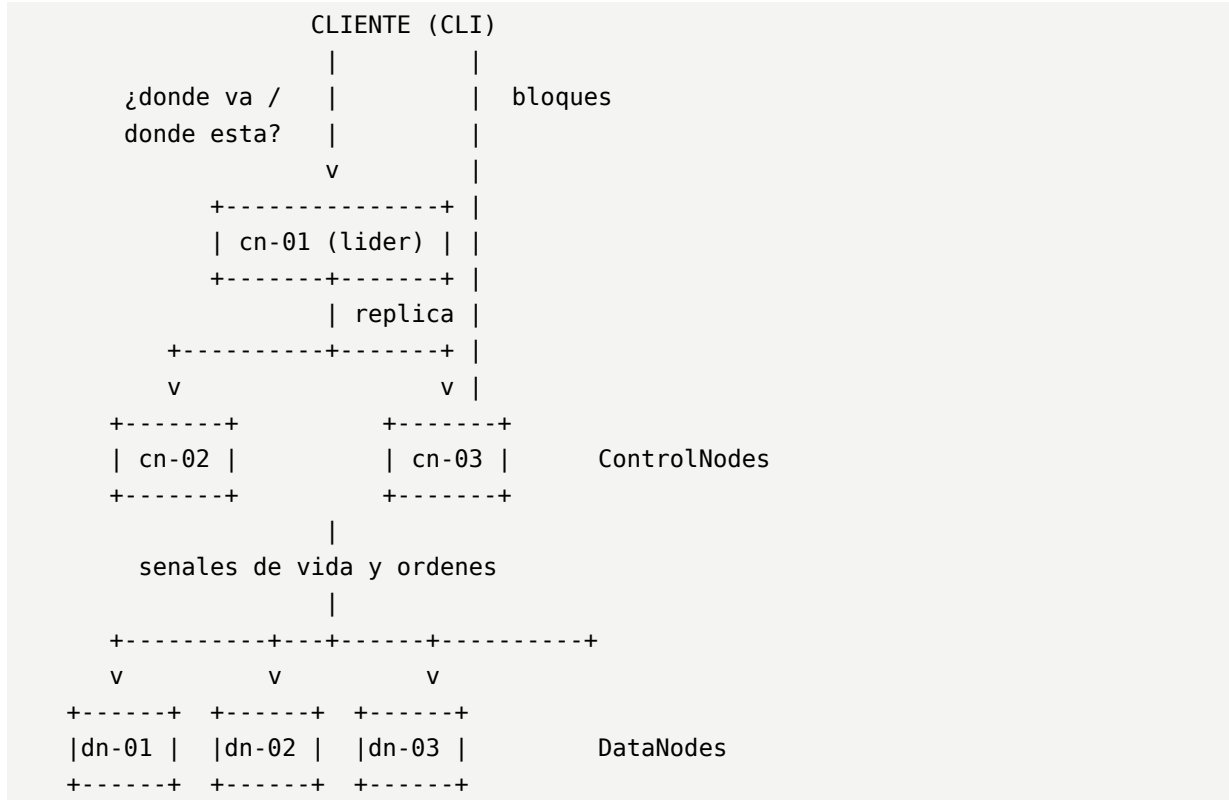
No se monta como unidad del sistema operativo, así que el acceso es por línea de comandos y API. No se modifica un archivo ya guardado: para cambiarlo se sube de nuevo. Un archivo lo escribe un solo cliente a la vez.

2. Arquitectura del sistema

Tipo de arquitectura: Maestro y Trabajador. El ControlNode coordina y sabe dónde está todo, mientras los DataNodes guardan los datos.

Se toma HDFS como referencia conceptual y no como implementación: no se instala ni se envuelve Hadoop, todo se construye desde cero. Lo que se hereda son dos patrones de la literatura, separar el plano de control del plano de datos y el modelo de escritura única. Lo propio, y donde el

diseño se aparta de HDFS, son cuatro decisiones. El tamaño de bloque sale de un modelo de dos restricciones y se adapta al archivo, en vez de ser un valor fijo por convención [N3]. El número de copias sale de un objetivo de disponibilidad [N4]. La ubicación usa una heurística de balanceo por ocupación, mientras HDFS ubica por cercanía de rack [N5]. Y el plano de control son tres nodos simétricos con elección de líder propia, en vez del esquema activo y pasivo con coordinadores externos que usa HDFS [N7].



Parámetros configurables

Ningún valor está escrito en el código. Todos se leen de un archivo de configuración al arrancar y se cambian sin recompilar, porque el tamaño del clúster y el objetivo de disponibilidad pueden variar.

Parámetro	Por defecto	Qué controla	Se deriva en
<code>block_size_max</code>	64 MB	Tamaño de bloque para archivos grandes	[N3]
<code>block_size_min</code>	8 MB	Piso del tamaño de bloque	[N3]
<code>blocks_per_node (k)</code>	4	Bloques mínimos por nodo que debe producir un archivo	[N3]
<code>replication_factor</code>	2	Copias de cada bloque	[N4]
<code>availability_target</code>	99.5%	Objetivo del que se recalcula el factor de replicación	[N4]

Parámetro	Por defecto	Qué controla	Se deriva en
<code>node_full_threshold</code>	90%	Ocupación a la que un nodo deja de recibir escrituras	[N5]
<code>rebalance_threshold</code>	10%	Desviación de ocupación que dispara el rebalanceo	[N5]
<code>heartbeat_interval</code>	3 s	Frecuencia de la señal de vida del DataNode	[N9]
<code>heartbeat_misses</code>	3	Señales perdidas para declarar muerto un nodo	[N9]
<code>blockreport_interval</code>	30 s	Frecuencia del reporte de bloques	no aplica
<code>client_parallel_blocks4</code>		Transferencias simultáneas del cliente	[N10]
<code>write_lease_ttl</code>	60 s	Vigencia de la reserva de escritura	no aplica
<code>cn_heartbeat_interval</code>	2 s	Latido entre ControlNodes	[N7]

Cómo se parte un archivo

El tamaño de bloque sale de resolver dos restricciones opuestas [N3]. La eficiencia exige bloques grandes, porque el costo de abrir cada conexión debe ser despreciable frente al tiempo de transferir el bloque. El reparto exige bloques pequeños, porque un archivo debe generar bloques suficientes para distribuirse entre todos los nodos. El intervalo donde ambas se cumplen es de 50 a 85 MB, y dentro de él se escoge **64 MB**.

Para archivos pequeños ese tamaño los dejaría en uno o dos bloques, así que el tamaño se calcula con:

```
B = clamp( block_size_min , S / (k · N) , block_size_max )
```

donde S es el tamaño del archivo y N el número de DataNodes vivos, redondeando a la potencia de dos inferior. Un archivo más pequeño que `block_size_min` queda en un solo bloque, y en ese caso no hay reparto que hacer porque la redundancia la sigue dando la replicación.

Sobre el tamaño máximo de archivo. El diseño no impone ninguno. Como el archivo se parte y sus bloques se reparten, ningún nodo necesita espacio para el archivo completo, así que lo único que limita es el espacio libre del clúster entero dividido por el número de copias, y eso crece agregando DataNodes sin cambiar nada del diseño. Los metadatos tampoco son un límite: un archivo de 1 TB produce unos 16.000 bloques, es decir cerca de 3 MB en el ControlNode [N2]. Si una carga no cabe en el espacio disponible, el ControlNode la rechaza de entrada en vez de dejarla fallar a mitad de camino.

Cada bloque lleva su número de orden y una firma SHA-256 de su contenido. El orden permite reensamblar el archivo exactamente, y la firma permite verificar al bajarlo que llegó completo y sin daños, además de servir como identificador del bloque. Un archivo solo es recuperable si todos sus bloques existen, están íntegros y se ordenan bien, así que orden, tamaño y firma se guardan

en los metadatos.

Cuántas copias se guardan

El número de copias sale de un modelo de disponibilidad [N4]. Se fija el objetivo de que un archivo esté disponible al menos el 99.5% del tiempo y se escoge el **mínimo** número de copias que lo alcanza. Con nodos al 99% de disponibilidad individual, ese mínimo es 2.

A eso se suma una restricción operativa: `replication_factor` no puede superar $N - 1$. Si fuera igual a N , todos los nodos tendrían todo y, al caerse uno, no quedaría destino válido donde rehacer la copia perdida. Con tres DataNodes esto también da 2.

El valor es configurable y la fórmula queda documentada precisamente para recalcularlo cuando cambie el clúster o el objetivo. La regla que nunca se rompe es que **las copias de un mismo bloque van siempre en DataNodes distintos**.

Cómo se decide en qué nodo va cada bloque

Ubicar óptimamente todos los bloques es un problema de balanceo de carga NP-difícil, así que se resuelve con una heurística [N5]: la **asignación voraz al nodo menos cargado**, conocida como *list scheduling* de Graham. Para cada bloque se elige el DataNode con menor porcentaje de disco ocupado, entre los que no tengan ya una copia de ese bloque y estén por debajo de `node_full_threshold`.

La heurística tiene una cota demostrada de $2 - 1/m$ respecto del óptimo, y aquí se comporta mejor que esa cota porque el peor caso de Graham necesita elementos de tamaños dispares, y todos los bloques de un archivo son iguales. Tiene además una ventaja práctica: como las escrituras siempre van al nodo más desocupado, **el clúster se equilibra solo con el uso normal**.

Quién hace la partición

El ControlNode no recibe archivos, porque administrar metadatos no es recibir 10 GB para cortarlos. La partición física la hace el cliente, pero el plan de partición y ubicación lo calcula el ControlNode, de modo que el cliente no necesita conocer ningún DataNode de antemano:

```
{
  "upload_id": "9f2c4a1e",
  "block_size": 67108864,
  "n_blocks": 16,
  "lease_expires_at": "2026-09-13T22:10:00Z",
  "blocks": [
    { "index": 0,
      "targets": [ {"node": "dn-02", "host": "10.0.1.12", "port": 8001},
                  {"node": "dn-03", "host": "10.0.1.13", "port": 8001} ] },
    { "index": 1,
      "targets": [ {"node": "dn-01", "host": "10.0.1.11", "port": 8001},
                  {"node": "dn-03", "host": "10.0.1.13", "port": 8001} ] }
  ]
}
```

El cliente recorre el archivo, corta cada bloque, calcula su firma y lo envía al primer destino de su lista. La lectura devuelve un plan simétrico, con las réplicas vivas de cada bloque.

Qué pasa al agregar un nodo

Un DataNode nuevo solo arranca y se registra. **No hay que reiniciar el clúster, ni tocar la configuración de los demás nodos, ni avisarle nada al cliente**. El ControlNode lo agrega a su lista y, como llega vacío, la heurística de escribir en el menos cargado hace que empiece a

recibir bloques de inmediato.

Los archivos ya guardados no se mueven solos, porque mover datos cuesta red y tiempo. El autobalanceo por escrituras corrige desviaciones pequeñas, pero no una grande: un nodo nuevo al 0% mientras los demás están al 50% tardaría en emparejarse lo que el clúster tarde en ingerir la mitad de su capacidad. Por eso, cuando la diferencia entre el nodo más lleno y el más vacío supera `rebalance_threshold`, el ControlNode ordena copiar bloques hacia el nuevo y borrar el original, en segundo plano y con límite de ancho de banda. Bajar el umbral da un clúster más parejo a costa de tráfico interno, y subirlo ahorra tráfico a riesgo de que un nodo se llene.

Qué pasa al quitar un nodo

Si se cae de golpe. Cada DataNode manda una señal de vida cada `heartbeat_interval` y se lo da por muerto tras `heartbeat_misses` señales perdidas, es decir 9 segundos [N9]. El ControlNode deja de ofrecerlo en los planes nuevos, marca como incompletos los bloques que tenía, y ordena a un nodo que sí los tiene que los copie a otro nodo sano, eligiendo destino con la misma heurística. Los clientes que estaban leyendo no fallan, porque su plan trae las direcciones de todas las copias y si una no responde van a la siguiente.

Si se retira a propósito. Deja de recibir escrituras, se copian sus bloques a los demás y solo cuando ninguno queda en riesgo se apaga. Así no hay un instante en que un archivo quede con una sola copia.

Si quedan menos nodos que copias configuradas. El sistema no se detiene: guarda las copias que puede, marca el bloque como incompleto, advierte en el registro que el factor excede los nodos disponibles y completa las faltantes cuando vuelva a haber nodos. Rechazar la escritura dejaría el servicio inutilizable justo cuando más se necesita.

Si hay accesos concurrentes durante todo esto. No se producen inconsistencias, porque los bloques son inmutables: copiar un bloque o borrar una copia sobrante no cambia su contenido, y quien esté leyendo una copia la termina sin verse afectado.

Qué pasa si se cae el ControlNode

Con un solo ControlNode, si se cae nadie sabría dónde está ningún bloque y el sistema quedaría inservible aunque los datos siguieran intactos. Por eso hay **tres**: uno líder y dos réplicas.

Los metadatos son persistentes. Cada cambio se anota en un registro en disco antes de confirmarse, y periódicamente se guarda una imagen completa del estado, así que un ControlNode que se reinicia recupera exactamente dónde estaba. Además, cada cambio se replica a las otras instancias antes de responderle al cliente, de modo que nada confirmado se pierde aunque el líder no vuelva nunca.

Si el líder deja de responder, las réplicas eligen quién toma el mando con el **algoritmo del matón**, modificado en dos puntos [N7]: gana el que tenga los metadatos más recientes, y se exige mayoría para proclamarse líder, lo que evita que una partición de red produzca dos líderes. Mientras tanto las réplicas atienden las consultas de solo lectura, que son la mayor parte del tráfico de control.

Cómo se mantiene la consistencia

El modelo es de **escritura única con publicación atómica** [N8]. Lo que garantiza en cada situación:

Situación	Garantía
Al escribir	Un solo cliente escribe un archivo a la vez, y la reserva vence sola si el cliente muere

Situación	Garantía
Al replicar	Los bloques son inmutables y se identifican por la firma de su contenido, así que dos copias no pueden divergir
Al leer	Toda lectura exitosa devuelve el archivo íntegro, verificado bloque a bloque contra su firma
Si falla un nodo	Las copias restantes son válidas sin reconciliación, y el sistema repone las faltantes en segundo plano
Al recuperarse	El ControlNode reconstruye su estado desde el registro, y un archivo a medio subir nunca fue publicado, así que no queda basura visible

3. Comunicaciones de red

Todo es **REST sobre HTTP con HTTPS**. El control viaja en JSON y los bloques viajan como contenido binario directo, sin codificar, para no inflar el tamaño ni cargarlos completos en memoria [N11].

Autenticación. El cliente se autentica una vez y recibe un token que envía en la cabecera *Authorization* de toda petición, tanto al ControlNode como a los DataNodes. Los nodos entre sí se identifican con una clave compartida del clúster en *X-Cluster-Key*.

Errores y reintentos. Toda operación de bloque es idempotente, porque el identificador del bloque es la firma de su contenido y reenviar un bloque que ya existe no lo duplica ni lo corrompe. Por eso los fallos de red se reintentan hasta 3 veces con espera creciente de 1, 2 y 4 segundos. Si un DataNode no responde en una lectura, el cliente pasa a la siguiente réplica sin reintentar. Si el plan venció, el ControlNode responde **410** y el cliente pide uno nuevo.

Protocolo por cada relación

Relación	Expone	Consume	Protocolo	Para qué
Cliente y ControlNode	ControlNode	Cliente	REST/JSON sobre HTTPS	Autenticación, comandos del sistema de archivos, planes de escritura y lectura, confirmación de subida
Cliente y DataNode	DataNode	Cliente	REST/binario sobre HTTPS	Subir y bajar bloques
ControlNode y ControlNode	ControlNode	ControlNode	REST/JSON sobre HTTPS	Replicar metadatos, latidos y elección de líder

Relación	Expone	Consume	Protocolo	Para qué
ControlNode y DataNode	ControlNode	DataNode	REST/JSON sobre HTTPS	Registro, señales de vida y reporte de bloques. En la respuesta viajan las órdenes de copiar o borrar
DataNode y DataNode	DataNode	DataNode	REST/binario sobre HTTPS	Propagar la copia al escribir y reponer copias al reparar

Cada nodo expone su propia API. El ControlNode escucha en el puerto **8000** y el DataNode en el **8001**.

API del ControlNode, la consume el cliente

Operación	Solicitud	Respuesta	Errores
POST /auth/login	{username, password}	200 {token, expires_in}	401
GET /fs/ls?path=	sin cuerpo	200 {entries:[{name, type, size, blocks, replicas_ok}]}	404, 403
POST /fs/mkdir	{path}	201	409 ya existe
DELETE /fs/rmdir?path=	sin cuerpo	204	409 no vacío
DELETE /fs/rm?path=	sin cuerpo	204	404, 403
POST /fs/mv	{src, dst}	200	404, 409
GET /fs/stat?path=	sin cuerpo	200 {size, block_size, n_blocks, replicas_ok, owner}	404
POST /files/upload/plan	{path, size}	200 con el plan de escritura	409 reserva activa, 507 sin espacio
POST /files/upload/commit	{upload_id, blocks:[{index, block_id, stored_on}]}	200 {status:"publicado"}	409 faltan copias, 410 reserva vencida
POST /files/upload/abort	{upload_id}	204	ninguno
GET /files/download/plan?path=	sin cuerpo	200 con el plan de lectura y las réplicas vivas	404, 503 bloque sin copias

```
POST /files/upload/plan -> { "path": "/andres/respaldo.tar", "size": 1073741824 }
200 OK                  -> { "upload_id": "9f2c4a1e", "block_size": 67108864,
                             "n_blocks": 16, "blocks": [ ... ] }
```

API del DataNode

Operación	La consume	Solicitud	Respuesta	Errores
PUT /blocks/{id}	Cliente, DataNode	Cuerpo binario, con cabecera Authorization y, si hay que propagar la copia, X-Forward-To	201 {block_id, size, stored_on}	401, 409 ya existe y se trata como éxito, 507
GET /blocks/{id}	Cliente, DataNode	Cabecera Authorization , y opcionalmente Range	200 binario, 206 parcial	401, 404
DELETE /blocks/{id}	ControlNode	Cabecera X-Cluster-Key	204	401, 404
POST /blocks/{id}/replicate	ControlNode	{target:{host,port}}	202	401, 404
GET /health	ControlNode	sin cuerpo	200 {node_id, capacity, used, n_blocks}	ninguno

```
PUT /blocks/a3f1...9c Content-Type: application/octet-stream
X-Forward-To: dn-03
<64 MB binarios>
201 Created          -> { "block_id": "a3f1...9c", "stored_on": ["dn-02","dn-03"] }
```

API interna del clúster

Operación	Expone y consume	Solicitud	Para qué
POST /cluster/register	Expone ControlNode, consume DataNode	{node_id, host, port, capacity}	Alta al arrancar
POST /cluster/heartbeat	Expone ControlNode, consume DataNode	{node_id, capacity, used}	Señal de vida. La respuesta trae órdenes pendientes
POST /cluster/blockreport	Expone ControlNode, consume DataNode	{node_id, blocks:[id]}	Detectar copias perdidas o sobrantes
POST /cluster/replicate	Entre ControlNodes	Entradas nuevas del registro	Replicar del líder a las réplicas
POST /cluster/election	Entre ControlNodes	{node_id, log_length}	Elegir nuevo líder

Cómo funciona una subida

1. El cliente se autentica y pide iniciar la carga, indicando ruta y tamaño.
2. El ControlNode valida permisos y espacio, toma la reserva, calcula tamaño y número de bloques, elige destinos y devuelve el plan. El archivo queda en curso, no publicado.
3. El cliente corta el archivo y envía cada bloque a su primer destino, hasta `client_parallel_blocks` en paralelo.
4. El DataNode que lo recibe lo guarda cifrado y lo propaga en cadena al siguiente destino, para que el cliente suba cada bloque una sola vez [N12].
5. El cliente confirma la carga enviando la lista de bloques y dónde quedó cada copia.
6. El ControlNode verifica las copias, anota el cambio en su registro persistente, lo replica y solo entonces publica el archivo. Si algo falló, la reserva vence, el archivo nunca se publica y los bloques sueltos se recolectan.

Cómo funciona una descarga

1. El cliente se autentica y pide el plan de lectura.
2. El ControlNode valida permisos y devuelve las réplicas vivas de cada bloque.
3. El cliente descarga en paralelo, y si una réplica no responde usa la siguiente.
4. Verifica la firma de cada bloque y los escribe en orden sobre el archivo de salida.
5. Si un bloque no tiene ninguna copia viva, el ControlNode responde con error y programa la reparación.

5. Infraestructura y entorno

Elemento	Definición
Nube	AWS Academy, instancias EC2 <code>t3.small</code> con Ubuntu 22.04
Máquinas	3 VMs , cada una con un contenedor ControlNode y uno DataNode, para un total de 6 nodos. Una cuarta VM queda preparada y apagada para demostrar en vivo la incorporación de un nodo [N14]
Contenedores	Docker con <code>docker compose</code> en cada VM
Red	Misma red privada, con los puertos 8000 y 8001 abiertos entre las VMs y el 8000 hacia el cliente
Disco	Un volumen de 20 GB por VM, montado en la carpeta de bloques del DataNode
Lenguaje de los nodos	Go , para ControlNode y DataNode [N13]
Lenguaje del cliente	Python , como CLI

VM	Contenedores
vm-01	cn-01, dn-01
vm-02	cn-02, dn-02
vm-03	cn-03, dn-03

VM	Contenedores
vm-04	dn-04, apagada para la demostración

Notas

[N1] **Por qué bloques y no objetos.** El almacenamiento por objetos trata el archivo como unidad indivisible. Bajo ese modelo el RNF4 solo se cumpliría con un particionamiento interno oculto al cliente, y el RNF5, que pide concurrencia dentro de un archivo, sería imposible de demostrar porque no habría forma de pedir una parte. El modelo por bloques cumple ambos directamente y es el que permite implementar **read** por rango de bytes.

[N2] **Costo de los metadatos.** Cada entrada de bloque ocupa del orden de 200 bytes. Con bloques de 64 MB, 1 TB de datos genera unos 16.000 bloques, o sea cerca de 3 MB de metadatos, una relación de 1 a 300.000. El ControlNode no es el límite de escalabilidad a esta escala, y lo sería en el orden de los petabytes, que es donde habría que particionar el espacio de nombres.

[N3] **De dónde sale el tamaño de bloque.** Se parte de dos parámetros del entorno: el caudal efectivo por conexión R de aproximadamente 100 MB/s, limitado por el volumen EBS de la instancia, y el costo fijo por bloque T_0 de unos 10 ms entre conexión TCP, negociación TLS, cabeceras y confirmación.

Restricción de eficiencia. El costo fijo debe ser a lo sumo una fracción f del tiempo útil de transferencia.

$$T_0 / (B/R) \leq f \quad \Rightarrow \quad B \geq T_0 \cdot R / f$$

Con $f = 2\%$ resulta $B \geq 50$ MB.

Restricción de reparto. Un archivo de referencia S debe dar al menos k bloques por cada uno de los N DataNodes, para que la heurística de ubicación tenga con qué balancear.

$$\text{techo}(S/B) \geq k \cdot N \quad \Rightarrow \quad B \leq S / (k \cdot N)$$

Con $S = 1$ GB, $N = 3$ y $k = 4$ resulta $B \leq 85$ MB.

El intervalo factible es entonces $50 \text{ MB} \leq B \leq 85 \text{ MB}$, y dentro de él se toma la potencia de dos: **64 MB**. Con ese valor el costo fijo es el 1.56% del tiempo de transferencia y un archivo de 1 GB da 16 bloques, algo más de 5 por nodo.

Esto explica por qué no se adoptan los valores habituales. Con 128 o 256 MB se viola el reparto, porque un archivo de 1 GB quedaría en 8 o 4 bloques y no habría distribución apreciable con tres nodos. Con 8 MB se viola la eficiencia, porque el costo fijo subiría al 12.5%. Para archivos pequeños se acepta esa ineficiencia a propósito, ya que en valores absolutos son milisegundos y el RNF4 pesa más que el rendimiento en ese rango.

[N4] **De dónde sale el número de copias.** Se modela cada DataNode como disponible de forma independiente con probabilidad a . Un bloque está indisponible solo si caen todos los nodos que lo tienen, y un archivo de M bloques necesita que todos estén disponibles:

$$\begin{aligned} A_{\text{bloque}} &= 1 - (1 - a)^{RF} \\ A_{\text{archivo}} &= A_{\text{bloque}}^M \end{aligned}$$

Con $a = 0.99$ y $M = 16$:

RF	Disponibilidad del archivo	Indisponibilidad	Costo
1	85.15%	14.85%	1x
2	99.84%	0.16%	2x
3	99.998%	0.002%	3x

El criterio es tomar el **mínimo RF que alcanza el objetivo**, fijado en 99.5%. Con 1 no se alcanza y con 2 sí. Con 3 se supera ampliamente, pero a cambio de un 50% más de almacenamiento para reducir una indisponibilidad que ya estaba por debajo del objetivo.

Encima se impone la restricción operativa de que RF no supere $N - 1$. Con RF igual a N , todos los nodos tienen todos los bloques y, al caer uno, no hay destino válido para rehacer la copia perdida, así que el tiempo de reparación se vuelve indefinido. Con $N = 3$ esto también fija $RF = 2$.

Ambos valores quedan configurables precisamente porque no existe un factor de replicación universal. Si el clúster crece o el objetivo sube a 99.99%, la misma fórmula da $RF = 3$.

[N5] Por qué esta heurística de ubicación. Asignar bloques a nodos minimizando la carga máxima es el problema de minimización del *makespan* en máquinas idénticas, que es NP-difícil, así que no se busca el óptimo sino una heurística acotada. Además los bloques llegan de a uno y hay que decidir en el momento, de modo que el algoritmo debe ser en línea.

Se escoge la **asignación voraz al menos cargado**, el *list scheduling* de Graham de 1966, con cota demostrada de $2 - 1/m$. Con $m = 3$, en el peor caso la carga máxima no pasa de 1.67 veces la ideal. Aquí el comportamiento real es mejor, porque el peor caso de Graham se construye con elementos de tamaños dispares y todos los bloques de un archivo son iguales.

Las alternativas y por qué se descartan:

- *Aleatoria*. No necesita estado, pero el desbalance esperado crece con la raíz del número de bloques y no mira la ocupación previa, así que un nodo que venía lleno se llena más.
- *Turno rotativo*. Es óptima solo si todos los nodos parten con igual ocupación y capacidad. Se rompe apenas entra un nodo o se borran archivos de forma desigual, que es lo que pasará en las pruebas.
- *Hashing consistente*. Es correcta cuando no hay mapa central y cada nodo debe calcular por su cuenta dónde está un bloque, y su virtud es que al entrar o salir un nodo solo se mueve una fracción $1/N$ de las llaves. Aquí el ControlNode ya tiene el mapa, así que esa virtud no aporta, y a cambio se pierde poder considerar el espacio libre real. Sería la opción correcta si la arquitectura asignada fuera la opción 2.

El umbral de rebalanceo va aparte del algoritmo porque resuelven cosas distintas. La heurística corrige el desbalance que generan las escrituras, no el que genera un cambio de topología, y mover datos ya escritos solo se justifica cuando el autobalanceo tardaría demasiado.

[N6] Por qué el cliente pide el plan cada vez. La alternativa sería guardar las direcciones de los DataNodes en la configuración del cliente, lo que obligaría a actualizar todos los clientes cada vez que un nodo cambie de dirección, entre o salga, y convertiría cualquier reconfiguración en una operación coordinada con los usuarios. Pidiendo el plan en cada operación, el cliente trabaja siempre con información vigente y nunca conoce la ubicación física de los bloques, que es lo que exige el RNF7.

[N7] Por qué el matón y no otro. Con un solo ControlNode todo depende de una máquina, y con dos el superviviente no puede distinguir entre que el otro murió y que él quedó aislado. Tres es el mínimo para que exista mayoría.

Se usa el **algoritmo del matón** de García-Molina, de 1982, con dos modificaciones. Gana el de metadatos más recientes en vez del de mayor identificador, porque lo que importa es no perder anotaciones confirmadas. Y se exige mayoría para proclamarse líder, lo que impide dos líderes simultáneos ante una partición.

Raft daría garantías más fuertes ante particiones, pero implementarlo completo, con términos, coincidencia de registros, instantáneas y cambios de membresía, no cabe en el tiempo del proyecto, y usar una implementación externa metería un componente que el equipo no podría sustentar. La *elección en anillo* usa $O(n)$ mensajes frente a $O(n^2)$, pero con $n = 3$ eso es 3 contra 9, una diferencia irrelevante, y el anillo se vuelve frágil si un nodo cae durante la elección.

La replicación de metadatos se hace antes de confirmar cada operación y no de forma periódica, porque replicar después dejaría una ventana en la que una operación ya confirmada al cliente solo existe en el líder. Si muere ahí, el sistema le habría mentido al cliente.

[N8] **Por qué escritura única.** Permitir modificar un archivo replicado obliga a resolver qué copia gana cuando dos escrituras concurrentes llegan a réplicas distintas, lo que exige quórum de lectura y escritura, vectores de versión y una política de reconciliación. Al declarar los bloques inmutables e identificarlos por la firma de su contenido, dos copias son idénticas por construcción y el problema desaparece. Es la misma simplificación deliberada de GFS y HDFS, y la que permite ofrecer consistencia fuerte en lectura sin implementar quórum. El costo se acepta explícitamente: no hay actualización parcial de archivos.

[N9] **De dónde salen los tiempos de detección.** El intervalo debe ser corto para detectar rápido y largo para no saturar. Con N nodos e intervalo t , el ControlNode recibe N/t mensajes por segundo, así que con 3 segundos y 100 nodos serían 33 por segundo, una carga despreciable. El intervalo no lo limita el ControlNode sino el tiempo de detección deseado.

Se declara muerto tras 3 intervalos perdidos y no tras uno porque un mensaje puede perderse por congestión momentánea, y declarar muerto a un nodo sano dispara copias innecesarias que son caras en red. Con tres intervalos, la probabilidad de falso positivo es el cubo de la de perder un mensaje. Los 9 segundos resultantes son menos de lo que tarda un cliente en agotar sus reintentos sobre una réplica, así que cuando el cliente se da por vencido el ControlNode ya sacó el nodo de los planes nuevos.

[N10] **De dónde sale el paralelismo.** Con 3 DataNodes, menos de 3 conexiones simultáneas deja nodos ociosos, y muchas más no aumentan el caudal porque el límite pasa a ser el enlace del cliente. Cuatro mantiene los tres nodos ocupados con una conexión de holgura para cubrir la latencia entre bloque y bloque. Se ajustará con las mediciones del Hito 4.

[N11] **Por qué REST y no gRPC.** REST es lo que sugiere el enunciado, lo que el equipo maneja y lo que permite probar cualquier operación con `curl` en la sustentación. gRPC daría serialización más compacta y multiplexación sobre HTTP/2, pero esas ventajas se miden en las cabeceras y en el número de llamadas, y aquí el tráfico lo dominan bloques de 64 MB donde las cabeceras son ruido. A cambio obliga a mantener definiciones aparte, generar código y depurar con herramientas extra.

Lo que sí es crítico es cómo viajan los bloques, como cuerpo binario y no dentro de un JSON. Meterlos en JSON obligaría a codificarlos en base64, lo que agrega un 33% de tráfico y fuerza a tener el bloque completo en memoria antes de enviarlo, justo lo contrario del objetivo del proyecto.

[N12] **Por qué la copia se propaga en cadena.** La alternativa es que el cliente suba cada bloque RF veces. El enlace de subida del cliente suele ser el tramo más lento de la ruta, así que eso multiplica por RF el tiempo de la operación que el usuario está esperando. Enviando el bloque una vez y dejando que el primer DataNode se lo pase al segundo, la segunda copia viaja por la red interna del clúster, que es más rápida, y el tiempo percibido no depende de RF .

[N13] Por qué Go. El enunciado orienta a Java o Go para los nodos y deja libre el cliente. Se escoge Go por tres razones ligadas a lo que el sistema hace. Un `DataNode` atiende muchas transferencias simultáneas y largas, y las gorrutinas arrancan con pilas de pocos kilobytes frente al orden de un megabyte de un hilo de la JVM, lo que permite sostener miles de transferencias sin administrar un pool. El binario es estático y da imágenes de contenedor de decenas de megabytes frente a varios cientos con la JVM, lo que importa en instancias de 2 GB donde conviven dos contenedores. Y no hay arranque en caliente ni recolector que afinar para obtener un rendimiento de entrada y salida estable.

El cliente va en Python, que el enunciado permite, porque su trabajo es leer archivos locales, cortarlos y hacer llamadas HTTP, donde la velocidad de desarrollo pesa más que el rendimiento bruto.

[N14] Por qué tres VMs con dos contenedores cada una. Seis nodos en tres máquinas distintas constituyen un sistema distribuido real sobre red, superan el mínimo exigido y ahorran la mitad del trabajo de aprovisionamiento frente a seis VMs separadas. Tiene además valor para la demostración, porque al apagar una VM caen a la vez un `ControlNode` y un `DataNode`, el escenario más exigente, ya que obliga simultáneamente a elegir líder y a reponer copias. La cuarta VM está apagada porque en la sustentación pueden pedir agregar un nodo en vivo.